SCQOS — Supreme Computation Kubernetes Admission Gate

SCQOS (Supreme Computation Quantum Operating System) is a pre-execution coherence webhook for Kubernetes.

It evaluates every `Pod`, `Deployment`, `Job`, `ConfigMap`, `Secret`, and `ServiceAccount` against nine coherence gates before Kubernetes admits it to the cluster.

This is not a patch to Kubernetes core. It is an official integration point: Kubernetes calls your gate; your gate decides.

How it integrates

Kubernetes exposes a built-in admission pipeline. Before a resource enters the cluster, the API server evaluates every registered admission controller and webhook:

```
Kubernetes API server
  → CreateAdmissionObjects()       — builds AdmissionReview
  → client.Post().Body(request)    — sends to /validate
  → r.Do(ctx).Into(response)       — receives your answer
  → VerifyAdmissionResponse()      — checks the response
  → if result.Allowed { admit }    — or ErrWebhookRejection
```

SCQOS operates as the external validating authority on the other side of `r.Do(ctx).Into(response)`.

Kubernetes asks: **“Can this state enter the cluster?”**
Supreme Computation answers: **`{ "allowed": false, "status": { "message": "SCQOS:Genesis:MISSING_OBSERVER" } }`**

Why SCQOS exists

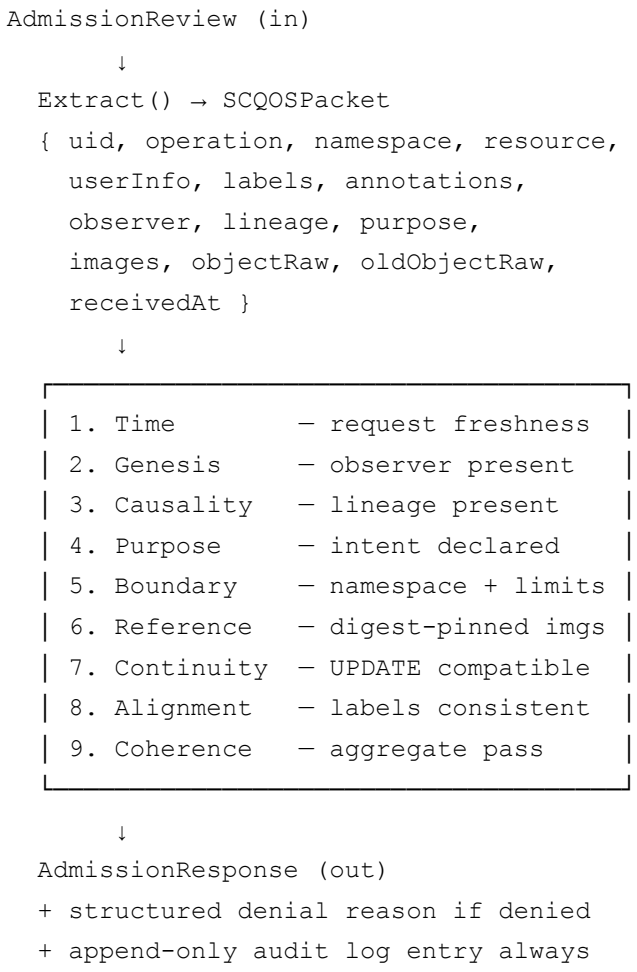
Traditional admission asks: **“Is this authorized?”**

SCQOS asks: **“Should this state exist at all?”**

Authorization without coherence still admits drift, orphaned lineage, unowned workloads, mutable references, and untraceable state transitions. SCQOS closes that pre-execution gap.

Gate sequence

Every admission request is normalized into a `SCQOSPacket` and evaluated through nine gates in order. First failure = deny. All nine must pass for admission.



Required annotations

Every resource admitted to a Supreme Computation-protected cluster must carry three annotations:

Annotation	Description	Example

`scqos.io/observer`	Accountable human or system actor	`alice` / `ci-bot`
`scqos.io/lineage`	Traceable cause: ticket, CI run, PR ref	`TICKET-42` / `https://ci.example.com/run/99`
`scqos.io/purpose`	Declared intent of this resource	`serve-api-traffic`

Missing any of these → **denied**.

Gate reference

#	Gate	Denial code	What it checks
1	Time	`REQUEST_TOO_OLD`	Request age ≤ 30 seconds
2	Genesis	`MISSING_OBSERVER`	`scqos.io/observer` present and non-empty
3	Causality	`MISSING_LINEAGE`	`scqos.io/lineage` present and non-empty
4	Purpose	`MISSING_PURPOSE`	`scqos.io/purpose` present and non-empty
5	Boundary	`DENIED_NAMESPACE` / `MISSING_RESOURCE_LIMITS`	Protected namespace; CPU+memory limits on all containers
6	Reference	`UNPINNED_IMAGE_REFERENCE`	All images pinned to `@sha256:` digest
7	Continuity	`NAMESPACE_DRIFT` / `KIND_DRIFT`	On UPDATE: namespace and kind are immutable
8	Alignment	`PURPOSE_LABEL_MISMATCH` / `MISSING_IDENTIFYING_LABEL`	Labels consistent with declared purpose
9	Coherence	—	Aggregate: always passes if gates 1–8 passed

Project structure

```
scqos-webhook/
├── go.mod
├── README.md
├── cmd/
│   └── webhook/
│       └── main.go              # TLS server, /validate endpoint
├── pkg/
│   ├── packet/
│   │   └── packet.go            # AdmissionReview → SCQOSPacket
│   ├── gates/
│   │   ├── gate.go              # Gate interface + GateResult + DefaultChain
│   │   └── core.go              # All nine gate implementations
│   ├── evaluator/
│   │   └── evaluator.go         # Fail-fast gate chain → AdmissionResponse
│   └── audit/
│       └── audit.go             # Append-only structured JSONL log
└── deploy/
    ├── namespace.yaml           # scqos-system namespace with exempt label
    ├── deployment.yaml          # 2-replica deployment + ServiceAccount
    ├── service.yaml             # ClusterIP service 443 → 8443
    └── webhook-config.yaml      # ValidatingWebhookConfiguration
```

Quickstart (kind cluster)

```
# 1. Clone and enter
git clone https://github.com/your-org/scqos-webhook
cd scqos-webhook

# 2. Replace module path throughout
find . -name "*.go" -exec sed -i \
  's|github.com/your-org/scqos-webhook|github.com/YOUR_ORG/scqos-webhook|g' {} +
sed -i 's|your-org|YOUR_ORG|g' go.mod

# 3. Resolve dependencies
go mod tidy

# 4. Start a local cluster
kind create cluster --name scqos-test

# 5. Generate TLS certificate
mkdir -p tls
```

```
openssl req -x509 -newkey rsa:4096 \
  -keyout tls/tls.key -out tls/tls.crt \
  -days 365 -nodes \
  -subj "/CN=scqos-webhook.scqos-system.svc"

# 6. Create namespace and TLS secret
kubectl apply -f deploy/namespace.yaml
kubectl -n scqos-system create secret tls scqos-webhook-tls \
  --cert=tls/tls.crt --key=tls/tls.key

# 7. Build and load image
docker build -t scqos-webhook:dev .
kind load docker-image scqos-webhook:dev --name scqos-test

# 8. Deploy webhook
kubectl apply -f deploy/deployment.yaml
kubectl apply -f deploy/service.yaml

# 9. Register webhook (with caBundle)
CA_BUNDLE=$(base64 -w0 tls/tls.crt)
sed "s|caBundle: \"\"|caBundle: \"${CA_BUNDLE}\"|" \
  deploy/webhook-config.yaml | kubectl apply -f -

# 10. Test — denied (no annotations)
kubectl run bare --image=nginx --restart=Never
# Error: admission webhook denied the request
# [SCQOS:Genesis:MISSING_OBSERVER] annotation "scqos.io/observer" is required

# 11. Test — allowed (annotated)
kubectl apply -f examples/valid-pod.yaml
```

Default posture

Setting	Value	Why
`failurePolicy`	`Fail`	Webhook down = cluster closed. No silent bypass.
`timeoutSeconds`	`5`	Slow gates are broken gates.
`sideEffects`	`None`	Webhook is read-only. Required for dry-run.
Namespace exempt		Bootstrapping escape hatch. Use

label	`scqos.io/exempt=true`	sparingly.

Audit log

Every decision — ALLOW and DENY — is written as a JSON line to `/audit/scqos-audit.log`.

```
{"timestamp":"2025-01-15T09:23:11.482Z","uid":"a3b2c1d0-...","operation":"CREATE"
```

The file is append-only. No entry is ever modified or deleted.

License

Apache 2.0